

SLIDE 1 — What are Arrays and Functions?

Definition:

A function can receive an **array as an argument** and process its elements.

Simple idea

Array → **Function** → **Process** → **Result**

Example:

```
void display(int a[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", a[i]);  
}  
int numbers[3] = {10, 20, 30};  
  
display(numbers, 3);
```

Output:

10 20 30

 **Remember: Array** → **Pass to Function** → **Process**

SLIDE 2 — Passing an Array to a Function

Structure

```
functionName(arrayName, size);
```

Example

```
void display(int a[], int n) {  
    for (int i = 0; i < n; i++)  
        printf("%d ", a[i]);  
}  
int numbers[5] = {10, 20, 30, 40, 50};
```

```
display(numbers, 5);
```

Explanation:

numbers → Array

5 → Number of elements

a[] → Receives the array

★ Remember

● Pass the **array name**

● Pass the **size separately**

SLIDE 3 — Using a Function to Calculate Sum

A function can perform calculations on an array.

Example

```
int sumArray(int a[], int n) {  
    int sum = 0;  
  
    for (int i = 0; i < n; i++)  
        sum += a[i];  
  
    return sum;  
}  
int numbers[4] = {10, 20, 30, 40};  
  
printf("%d", sumArray(numbers, 4));
```

Explanation:

10 + 20 + 30 + 40

↓
100

Output:

🧠 Remember: Function can process the entire array

SLIDE 4 — Why Use Functions with Arrays?

Functions make array programs **simple and reusable**.

🎓 Example

One function can calculate the sum:

```
sumArray(numbers, 5);
```

The same function can work with another array:

```
sumArray(marks, 5);
```

Benefits

- **Reusable** → Use the function multiple times
- **Organized** → Keeps code neat
- **Easy to understand** → Separate tasks
- **Easy to modify** → Change one function

🧠 Basic Pattern

CREATE ARRAY → PASS ARRAY → FUNCTION PROCESSES → RESULT

SLIDE 5 — RECAP

Array + Function = Reusable Code! 

Flow

CREATE ARRAY

↓
PASS ARRAY
↓
FUNCTION
↓
PROCESS ELEMENTS
↓
RESULT

5 Things to Remember

1. Arrays can be passed to functions
2. Pass the array name to the function
3. Pass the array size separately
4. Functions can process array elements
5. Functions make code reusable

Final Example

```
int findSum(int a[], int n) {  
    int sum = 0;  
  
    for (int i = 0; i < n; i++)  
        sum += a[i];  
  
    return sum;  
}  
int numbers[3] = {5, 10, 15};  
  
printf("%d", findSum(numbers, 3));
```

Output:

30

Memory Trick

ARRAY → PASS 

FUNCTION → PROCESS 

RETURN → RESULT 

QUESTIONS

1. How can an array be passed to a function?

- A) Pass the array name
- B) Pass only the first element
- C) Pass the data type
- D) Pass the loop

 **Hint:** The array name represents the array when passed to a function.

 **Glossary: Argument** → A value passed to a function.

 **Answer: A) Pass the array name**

2. Why is the array size usually passed separately?

- A) To delete the array
- B) To tell the function how many elements to process
- C) To change the data type
- D) To stop the function

 **Hint:** The function needs to know how many elements to loop through.

 **Glossary: Size** → Number of elements in an array.

 **Answer: B**

3. What is the output?

```
int sumArray(int a[], int n) {  
    int sum = 0;  
  
    for (int i = 0; i < n; i++)  
        sum += a[i];  
  
    return sum;  
}
```

```
int numbers[3] = {10, 20, 30};
```

```
printf("%d", sumArray(numbers, 3));
```

 **Hint:** Add all three elements.

 **Glossary: Return** → **Send a result back from a function.**

 **Answer: 60**

4. Fill in the blank

Complete the function call:

```
int numbers[4] = {10, 20, 30, 40};
```

```
display(_____, 4);
```

 **Hint:** Pass the array name.

 **Glossary: Array name** → **The identifier used to refer to the array.**

 **Answer: numbers**

5. What can a function do with an array?

- A) Only create it
- B) Only delete it
- C) Process its elements
- D) Nothing

 **Hint:** Functions can perform operations such as sum, search, and display.

 **Glossary: Process** → **Perform an operation on data.**

 **Answer: C**

6. Arrange the steps in the correct order:

- A. Function processes elements

- B. Create an array
- C. Pass the array
- D. Get the result

 **Hint:** Create → Pass → Process → Result.

 **Glossary: Result** → The final output produced by the function.

 **Answer: B → C → A → D**

7. Which parameter receives the array in this function?

```
void display(int a[], int n) {  
    printf("%d", a[0]);  
}
```

- A) `n`
- B) `a[ ]`
- C) `display`
- D) `printf`

 **Hint:** Look for the parameter with square brackets.

 **Glossary: Parameter** → A variable that receives a value passed to a function.

 **Answer: B) `a[ ]`**